

P3831R0

Contract Labels Should Use Annotation Syntax

Published Proposal, 2025-09-07

This version:

<https://wg21.link/P3831R0>

Author:

[Yihe Li](#)

Audience:

SG21

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Target:

[\[P3400R1\]](#)

Source:

github.com/Mick235711/wg21-papers/blob/main/P3831/P3831R0.bs

Issue Tracking:

[GitHub Mick235711/wg21-papers](#)

Table of Contents

1 Revision History

1.1 R0 (2025-09 Mailing)

2 Background and Motivation

2.1 Contract Labels

2.2 Annotations

2.3 Prior Arts

3 Design

3.1 Proposal 1: Assertion-Control Objects

3.2 Proposal 2: Assertion Using Directives

3.3 Proposal 3: Assertion-Combination Operator

3.4 Proposal 4: Access-Control Object on Violation

3.5 Proposal 5: Ambient-Control Objects

3.6 Controllable Properties

3.7 Possible Issues

4 Suggestions

References

Normative References

Informative References

Contract labels ([\[P3400R1\]](#)) are one of the most important extensions proposed to C++26 Contracts, providing the ability to control the behavior of specific contract assertions. This proposal argues that instead of inventing a new syntax for parameterizing contract assertions, the labels should utilize the existing feature in the standard that permits this parameterization with defined semantics, namely annotations ([\[P3394R4\]](#)).

§ 1. Revision History

§ 1.1. R0 (2025-09 Mailing)

- Initial revision.

§ 2. Background and Motivation

§ 2.1. Contract Labels

Contracts ([P2900R14]) were voted in as one of the most important features for C++26 back in Hagenberg (2025-02); however, several features are still missing from the MVP that are present in the standard. One of the most important add-on features is the ability to control the flexibility of Contracts in source code, such as controlling the available evaluation semantics of a particular contract assertion. The proposal [P3400R1] tries to address this lack of functionality by introducing the concept of *contract labels*, which includes a way of using *assertion-control objects* with the syntax `pre<my_label>(expression)` to control the behavior of contract assertions. To use the examples present in [P3400R1]:

```
struct my_label_t {};  
  
constexpr my_label_t my_label;  
  
int f(int i)  
    pre<my_label>(i > 0)  
    post<my_label>(r: r > 0)  
{  
    contract_assert<my_label>(i > 0);  
    // ...  
}
```

`my_label` would then be used to customize the properties of the contract assertion.

The initial motivation for this syntax is to simulate the template type parameter specification syntax, as the labels are also a way of parameterizing contract assertions, exactly like template type parameters are a way of parameterizing template functions or classes. The assertion-control objects are also required to be produced by constant expressions to further enforce this similarity. However, the author of this proposal identifies several problems with this piggyback syntax approach:

- The above declaration looks very much like a function call to a function declared with signature `template<auto V> void pre(bool b)` (with proper constraints), which is presumably the design intent of contract labels. However, such a function call will require the type of the assertion-control objects to be structural in order for them to be passable as constant template parameters, which is clearly not the case in contract labels. In the latter scenario, the only requirements for these objects are that their type is a class type (the concept `assertion_control_object` in Section 2.1.1 of [P3400R1]). Using the same syntax for both while having different requirements on types will be very confusing for the users:

```
template<std::contracts::labels::assertion_control_object auto V>  
void pre(bool b);  
  
struct A { constexpr A() = default; };  
struct B : private A { constexpr B() = default; }; // literal, but not structural  
inline constexpr B b;  
  
void use()  
{  
    pre<b>(true); // ill-formed: not structural  
}  
  
void fun() pre<b>(true); // well-formed?
```

- The exact semantics of this new syntax will need to be manually defined. For example, [P3400R1] explicitly leaves unspecified whether the same assertion-control object is used for all instances of the same contract assertions, and requires them not to be in the immediate context (not subject to SFINAE). This in itself is not a problem, but it can be a

burden to maintain a separate list of semantics for this narrowly utilized new syntax and fully explore its interaction with the rest of the language.

- The API design for fetching the assertion-control objects attached to a contract assertion (Proposal 4) is very clumsy; [P3400R1] provides the `void* contract_violation::control_object()` function that fetches a pointer to the object, but also admits that such a pointer is basically useless and highly dangerous due to the lack of type information. The proposal tries to mitigate this issue by requiring that this function only return a non-`nullptr` pointer when the control object is polymorphic, and relying on `dynamic_casts` to recover the correct types. However, the proposal fails to provide an actual use case of polymorphic control objects. Since they are required to be constant objects, polymorphism will be of very limited usefulness. (To that end, why is this function not marked `constexpr` or even `constexpr?`)
- The concept of *ambient-control objects* (Proposal 5) intends to automatically attach the same label to (for example) all contract assertions on functions declared within a particular namespace. This is intended to be achieved through a kind of implicit control-object declaration syntax invoked via `contract_assert implicit <expression>;` that injects the label to all declarations in the same scope. However, several important design decisions remain unanswered for such declarations. See the sections below for more on this.

In the end, contract labels at their core are yet another way to come up with a syntax to annotate declarations in a program-detectable manner to adjust the behavior of the program, in other words, a non-ignorable attribute syntax. Such experiments have been proposed several times ever since attributes were introduced in C++11 and immediately failed to be the supposed solution to the decoration problem (see [attribute-critics] for more on this topic). However, the standard now has a solution to this problem.

§ 2.2. Annotations

Annotations, proposed by [P3394R4], were initially intended as a complement to the reflection proposal ([P2996R13]) to provide a way to annotate declarations such that they can be read by reflection queries. Such an annotation has significant usefulness, such as providing parser arguments in a command-line argument parser, providing test case parameterization, and providing a general way of customizing class derivations similar to the Rust `#[derive]` macro.

Initially, in the early days of reflection, people were trying to achieve this functionality with the same means that contract labels are trying to imitate: template arguments. Specifically, the workaround was to try to use a template parameter pack to provide the additional annotations:

```
template <typename T, auto... Annotations>
using Noted = T;

struct C
{
    Noted<int, 1> a;
    Noted<int*, some, thing> b;
};
```

However, it is quickly realized that annotation can be much more useful than just annotating template classes or functions, coupled with the difficulties encountered by trying to design a reflection API to retrieve the `Annotations` (also an obstacle to the contract labels design, as shown above), leading to the invention of a new syntax for annotation: `[[=expr]]`:

```
struct C
{
    [[=1]] int a;
};
```

Such an approach leverages the syntax space left behind in the attribute syntax instead and has significant advantages over the workaround:

- Since the attribute syntax is reused, a lot of the places where annotations can be placed are already defined by the standard for attributes, eliminating the need to manually define them again.
- Annotations are conceptually a non-ignorable attribute, so using similar syntax makes sense.

- Designing a reflection query to retrieve annotations is significantly easier, and can also be coordinated with attribute reflection efforts ([P3385R5]) to provide a more consistent API overall.

After [P3394R4]’s adoption into the C++26 standard in Sofia (2025-06), we finally have a ready-made solution to the decoration problem. Therefore, to avoid duplication of efforts, contract labels should also utilize annotations instead of inventing yet another new syntax.

§ 2.3. Prior Arts

The idea of adding a label to contracts is not new. For instance, C++20 Contracts ([P0542R5]) use the concept of *modifiers* to specify contract assertions’ assertion levels, which can be seen as a more narrow form of contract labels. The syntax for modifiers also utilizes the syntax space left over by the attribute syntax:

```
int f(int x)
  [[expects audit: x > 0]]
  [[ensures axiom res: res > 1]];
```

This proves that the idea of treating such annotations as a kind of attributes is already present back in the C++20 Contracts design cycle, and thus, utilizing new annotation syntax can be seen as a continuation of this thinking.

The similarities between several "annotation-like" proposals have also been highlighted by [P3661R0], which tries to unify the syntax of Profiles ([P3589R2]), contract labels, and reflection annotations by pointing out their core similarities. This proposal can be seen as a concrete and actionable version of suggestions regarding contract labels in [P3661R0].

§ 3. Design

The overall design proposed by this proposal is to replace the `pre<object>(expr)` syntax proposed by [P3400R1] with the annotation syntax, i.e., `pre [[=object]] (expr)`. See the table below for a comparison of two syntaxes:

[P3400R1] syntax	This Proposal
<pre>struct my_label_t {}; constexpr my_label_t my_label; int f(int i) pre<my_label>(i > 0) post<my_label>(r: r > 0) { contract_assert<my_label>(i > 0); // ... }</pre>	<pre>struct my_label_t {}; constexpr my_label_t my_label; int f(int i) pre [[=my_label]] (i > 0) post [[=my_label]] (r: r > 0) { contract_assert [[=my_label]] (i > 0); // ... }</pre>
<pre>struct my_label_2_t {}; constexpr my_label_2_t my_label_2; void g(int i) pre<my_label my_label_2>(i > 0);</pre>	<pre>struct my_label_2_t {}; constexpr my_label_2_t my_label_2; void g(int i) pre [[=my_label my_label_2]] (i > 0); // or, optionally pre [[=my_label, =my_label_2]] (i > 0);</pre>

The rest of this section will discuss each proposal in [P3400R1], and how they should be redesigned with the introduction of annotations.

§ 3.1. Proposal 1: Assertion-Control Objects

The grammar for the new *assertion-control-specifiers* actually occupies the same syntax as the attributes, directly prepending the latter. Since [P3394R4] treated annotations as a kind of attributes, we automatically get the ability to use annotations in `pre`, `post`, and `contract_assert` for free due to the existence of *attribute-specified-seqs* in their grammar (introduced by [P3088R1]), thus eliminating the need to modify their grammar tree at all. However, several design decisions inherited from the annotation design are worth highlighting:

- A last-minute change made in Sofia (2025-06) to [P3394R4] required that annotations and attributes must be specified separately; mixing them like `[[nodiscard, =2]]` is no longer allowed.
- For `contract_assert` only, [P3088R1] made the distinction between attributes that are specific to assertion statements (such as vendor-specific labels) and attributes that can pertain to any statement (such as `[[likely]]`). The former is required to be in the infix position after the `contract_assert` keyword, and the latter is required to be in the prefix position before the keyword, just like any other statements. This coincides nicely with the annotation design, as vendor-specific attributes that modify the contract assertion semantics can also be seen as a kind of contract labels. Making them occupy the same grammatical position seems desirable and natural.
- Due to the need to reflect on annotations, their type is required to be structural. This is **not** true for contract labels in [P3400R1], but the author believes that this is a reasonable restriction, as the control objects being suggested as examples in [P3400R1] all have structural types anyway.
- [P3400R1] further proposes that the type of controlling objects must satisfy the `assertion_control_object` concept, for which annotations are not required. However, this can be mitigated by requiring all annotations appearing in a contract assertion to satisfy this concept, too.
- The design decisions made in [P3400R1] regarding name mangling and immediate context are also no longer relevant, as annotations already do not participate in name mangling and SFINAE.

§ 3.2. Proposal 2: Assertion Using Directives

[P3400R1] went on to propose a new kind of using declaration and directive, the `contract_assert using` declaration and directives, aiming to be a way to minimize redundant namespaces for labels. Basically, prepending the `contract_assert` keyword to any using declaration/directive makes them only apply to contract labels, and does not introduce the relevant names to other usages in the scope.

Annotations actually used to have a similar functionality inherited from attributes: the attributes `using` prefix ([P0028R4]).

```
[[using clang: A, B, C]]  
// equivalent to  
[[clang::A, clang::B, clang::C]]
```

Essentially, the existence of `using N:` before the attribute list makes every attribute following implicitly adopt the `N::` prefix, as a way to ease the attachment of multiple vendor-specific attributes. However, this prefix is not permitted to be used with annotations, which seems to the author of this proposal to be a design oversight. Originally, this exclusion is a must in [P3394R3] or earlier since attribute prefixes and annotation namespace prefixes are entirely different, and mixing them is nonsensical:

```
[[using clang: amdgpu_waves_per_eu, =nick("weapon")]]  
// does this refer to clang::nick?
```

However, after the last-minute decision to separate attributes and annotations, the possibility of confusion was eliminated, but this design decision was not revisited. The author of this proposal thinks that allowing a `using` prefix here (that functions as if a using directive is used) will also solve the contract label problem partially:

```
[[using std::contracts::labels: =label_a, =label_b]]
```

The author does admit that this is not a full solution, as this syntax requires a separate `using` prefix before each contract assertion. Therefore, no changes are proposed in this regard for now. However, the author still has reservations about the usefulness of introducing a new kind of using declarations just for such a narrow context.

§ 3.3. Proposal 3: Assertion-Combination Operator

To allow combining labels with orthogonal purposes, [P3400R1] introduced the concept of using `operator` to create new combined labels. This in itself is fine, and can be directly ported to the annotation syntax, as the syntax allows arbitrary constant expressions too:

```
<my_label | my_label_2>
[ [=my_label | my_label_2] ]
```

However, the author of this proposal would like to point out that allowing only one argument in the template argument-like syntax is a bit odd, as template arguments are naturally multi-argument. Furthermore, the annotations API assumes that multiple annotations can be used, similar to attributes. Therefore, the author also suggests allowing multiple annotations here and letting the violation handler decide how to combine them. The default implementation of the handler can do an `operator` on all the arguments implicitly, while custom handlers have the freedom to interpret them differently, such as logging all the labels involved.

As a further note, [P3394R4] also listed standard functions combining annotations as a future direction worth pursuing; perhaps `operator` can be delayed to wait for a more general facility to combine all kinds of annotations, instead of just contract labels.

§ 3.4. Proposal 4: Access-Control Object on Violation

The API proposed in [P3400R1] for retrieving access-control objects has several limitations as argued above: they are untyped, and can only work with polymorphic types. Moreover, they assume that there is only one object specified for each contract assertion.

Fortunately, by adopting the annotation syntax, we no longer need to come up with a new API design; we can directly reuse the reflection API to retrieve annotations. Originally, the author considered extending the `std::meta::annotations_of` function such that it can be applied to the reflection of the contract violation object, and the usage will be:

```
void handle_contract_violation(const contract_violation &violation)
{
    constexpr auto annotations = std::define_static_array(
        std::meta::annotations_of(std::meta::reflect_constant(violation))
    );
    // Use annotations...
}
```

However, this is both incredibly verbose and also requires `contract_violation` to be a structural type. On its own, this is not a problem since the only APIs specified for that type are several member functions; however, this prohibits the implementation from providing private members to store the location and comments, etc.. Therefore, for ease of use and to permit more flexibility, the author suggests providing a new magic member function `constexpr std::vector<std::meta::info> annotations() const noexcept` to the contract violation class, with the return value magically filled with the annotations' reflections.

The following table showcases the new API design:

[P3400R1] API Design	Proposed API Design
<pre> namespace std::contracts { class contract_violation { public: // ... void *control_object() const noexcept; // ... } } </pre>	<pre> namespace std::contract { class contract_vio { public: // ... consteval std:: // ... } } </pre>
<pre> struct my_dynamic_tag { constexpr my_dynamic_tag() = default; constexpr virtual ~my_dynamic_tag() = default; }; inline constexpr my_dynamic_tag my_dynamic_label; void handle_contract_violation(const contract_violation& violation) { if (auto* dynamic_tag = dynamic_cast<my_dynamic_tag*>(violation.control_object()) != nullptr) { std::cout << "Dynamic Tag!\n"; } else { std::cout << "No Dynamic Tag\n"; } } void f() pre<my_dynamic_label>(false) // prints "Dynamic Tag!" pre(false); // prints "No Dynamic Tag" </pre>	<pre> struct my_dynamic_tag { constexpr my_dynam }; inline constexpr my_dyn void handle_contract_v { constexpr auto anno template for (const { if constexpr (s std::meta::)) { std::cout < return; } } std::cout << "No Dy } void f() pre<my_dynamic_labe pre(false); // pri </pre>
<pre> namespace std::contracts { template<typename T> T* get_constituent_label(void* control_object); } </pre>	<pre> namespace std::contract { consteval std::vect std::meta::info // for convenience, template <std::meta consteval std::vect R&& labels, stc } } </pre>

§ 3.5. Proposal 5: Ambient-Control Objects

[P3400R1] identifies that often, all declarations in a scope may want to attach the same contract labels. To achieve this, the proposal introduces implicit control-object declarations in the form of `contract_assert implicit <label>;` (we can change `<label>` to `[[=label]]` for consistency) that automatically add `label` to all the contract assertions in the same scope. However, there are several design decisions unanswered for this facility:

- Is the label prepended to all contract assertions' label list, or appended to it? This can be significant if `operator|` is not symmetric.
- Why are multiple declarations prohibited? Injecting multiple labels is clearly useful.
- Is it possible to "inherit" base class/enclosing namespace's implicit contract labels?

Moreover, [P3394R4] pointed out that a more general facility to let annotations have callbacks that are invoked by the declaration has great usefulness. Such a facility, combined with advanced code generation functionalities, can actually give us the implicit labels for free; just attach an annotation to the namespace/class that automatically goes through all the declarations and modifies them to add the labels. The correct way to achieve implicit labels is thus perhaps providing a `[[=attach(label1, label2)]]` instead of a new kind of statement. As a result, the author recommends delaying the adoption of implicit label statements in anticipation of a more general annotation callback facility.

§ 3.6. Controllable Properties

The rest of [P3400R1] is not directly relevant to this proposal, as they do not touch the syntax of labels. However, SG21 may want to consider changing to a design based on metafunctions instead for things like identification labels; although using the proposed concept with a splicer is possible, doing it directly may be more beneficial.

§ 3.7. Possible Issues

The main issue that the author foresees with this proposal is the fact that relying on annotation syntax puts a hard dependency on reflection for Contracts; `<contracts>` may even need to include `<meta>` (or move the proposed metafunctions there?), and fetching annotations will become very template/reflection-heavy. However, labels are conceptually the same as annotations, and annotations are inherently a reflection-based feature, so such dependency is probably inevitable.

§ 4. Suggestions

In conclusion, this proposal suggests the following changes to [P3400R1]:

1. Use annotation syntax `pre [[=label]] (expr)` instead of `pre<label>(expr)`.
2. Allow multiple annotations to attach to a contract assertion, and optionally remove `operator|` while waiting for a more general annotation combination facility. The combination logic can instead be represented in the default implementation of the contract violation handler.
3. Adopt the `using` prefix for annotations, and remove the `contract_assert using` declarations.
4. Provide a `consteval vector<info> annotations()` member function instead of the unsafe `void*`-returning member function, and optionally provide metafunctions to get labels with a specific type.
5. Remove implicit label statements, and wait for a more general annotation callback facility.

§ References

§ Normative References

[P2900R14]

Joshua Berne, Timur Doumler, Andrzej Krzemiński. *Contracts for C++*. 14 February 2025. URL: <https://wg21.link/p2900r14>

[P3394R4]

Daveed Vandevoorde, Wyatt Childers, Dan Katz, Barry Revzin. *Annotations for Reflection*. 20 June 2025. URL: <https://wg21.link/p3394r4>

[P3400R1]

Joshua Berne. *Specifying Contract Assertion Properties with Labels*. 28 February 2025. URL: <https://wg21.link/p3400r1>

§ Informative References

[ATTRIBUTE-CRITICS]

Barry Revzin. *On the Ignorability of Attributes*. URL: <https://brevzin.github.io/c++/2025/03/25/attributes/>

[P0028R4]

J. Daniel Garcia, Daveed Vandevoorde. *Using attribute namespaces without repetition*. 22 June 2016. URL: <https://wg21.link/p0028r4>

[P0542R5]

J. Daniel Garcia. *Support for contract based programming in C++*. 8 June 2018. URL: <https://wg21.link/p0542r5>

[P2996R13]

Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. *Reflection for C++26*. 20 June 2025. URL: <https://wg21.link/p2996r13>

[P3088R1]

Timur Doumler, Joshua Berne. *Attributes for contract assertions*. 13 February 2024. URL: <https://wg21.link/p3088r1>

[P3385R5]

Aurelien Cassagnes. *Attributes reflection*. 19 May 2025. URL: <https://wg21.link/p3385r5>

[P3394R3]

Daveed Vandevoorde, Wyatt Childers, Dan Katz, Barry Revzin. *Annotations for Reflection*. 17 May 2025. URL: <https://wg21.link/p3394r3>

[P3589R2]

Gabriel Dos Reis. *C++ Profiles: The Framework*. 19 May 2025. URL: <https://wg21.link/p3589r2>

[P3661R0]

Andrzej Krzemieński. *Attributes, annotations, labels*. 15 March 2025. URL: <https://wg21.link/p3661r0>